Efficient Embedded AI Deployment for Real-Time Card Recognition

Gregory Wagonblast Harsh Selokar
Department of Electrical and Computer Engineering
University of Pittsburgh
Pittsburgh, PA 15260
{ghw3, hjs32}@pitt.edu

Abstract

This paper presents a specialized TinyML framework for the real-time detection of Magic: The Gathering (MTG) cards on highly resource-constrained hardware. Using the Arduino Nano 33 BLE Sense (1) and an OV7675 (2) camera sensor, we developed a 2D convolutional neural network (CNN) optimized via 8-bit integer quantization. To address the scarcity of environment-specific training data, we utilized a hybrid augmentation strategy combining Scryfall bulk data with synthetic, camera-realistic negative samples. Our final model occupies only 82 KB of memory and achieves 99.45% accuracy with ~359.1 ms inference time. This demonstrates that robust, vision-based object classification is achievable on embedded devices with limited SRAM and no dedicated hardware acceleration.

1 Introduction

The deployment of efficient deep learning models on embedded devices presents a significant engineering challenge due to the resource-constrained nature of edge hardware. Standard computer vision architectures, such as YOLO or ResNets, typically require megabytes of SRAM and significant computational throughput, making them incompatible with platforms such as the Arduino Nano 33 BLE Sense (1). This project addresses these limitations by developing a low-cost, portable solution to automate the recognition of MTG collections. Using TinyML techniques, we demonstrate that a highly optimized model can perform real-time card recognition on a device with only 256 KB of SRAM and 1 MB of Flash memory, providing a scalable alternative to expensive, PC-dependent scanning hardware.

2 Related work

This research builds upon the foundational person-detection methodology described in *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers* (3). The implementation utilizes the TensorFlow Lite for Microcontrollers (TFLM) framework (4), which enables the execution of machine learning models on bare-metal hardware by using a limited set of optimized kernels. While contemporary TinyML vision benchmarks primarily focus on general tasks such as binary person detection, our work adapts these concepts for high-granularity object classification. Specifically, we transition from binary detection to the differentiation of complex card art from varied negative samples.

Preprint.

3 Methodology

The implementation followed a rigorous pipeline of data synthesis, model architecture optimization, student/teacher training, and post-training quantization.

3.1 Data acquisition and synthesis

To ensure robust classification, we curated a balanced data set consisting of two classes. For the `default_cards` class, we developed a multi-threaded down loader to fetch 50,000 high-quality images via the Scryfall API (5). To minimize false positives in a home environment, we generated 150,000 synthetic non-card images.

Positive samples We download the `default_cards` bulk data file then extract the highest-resolution image for each card, and only handle the front side of dual faced cards. This is only possible because MTG tends to prefer a standard format for the front and back. To improve download times, all downloads are parallelized across 16 threads. These images are clean, full-frame, and orthogonally photographed. These do not match the real world and will require augmentation; this will be addressed later.

Negative samples The `not_card` class is constructed from two complementary sources with a 80/20 ratio. The bulk of negatives (80%) are sampled from the COCO 2017 dataset (6) via the HuggingFace `datasets` streaming API, providing about 120,000 real-world photographs of everyday objects and scenes. Furthermore, another 30,000 images are procedurally generated; these images are surface textures, document-like content, partial occlusions, poor focus, lens glare, etcetera.

Augmentations Some augmentation is imprinted over the photos to improve the robustness of the model. Considerations taken with the model include downloading a second set of photos to overlay the Scryfall images over. This simulates real world environments in which the cards may exist such as a room or a shop. A short list of the augmentations are below:

- **Geometric augmentations** simulate camera viewpoint and framing variation:
  - **Perspective warp**: a four-point perspective transform applied with one of four randomly chosen styles simulating cards viewed at non-orthogonal angles.
  - **Random crop with oversize**: images are resized to 1.0–1.15$\times$ the target dimension and randomly cropped back.
  - **Card-on-background compositing**: each positive sample is pasted onto a random COCO background at 25–80% scale at a random in-frame position, simulating the range of card sizes the camera observes at different distances and angles.

  Photometric augmentations simulates lighting and white-balance variation:
  - **Random brightness**: pixel intensities shifted by up and down to $\pm 0.35 \times 255$, coveringlighting levels of dim to bright.
  - **Random contrast**: scaling factor sampled uniformly from $[0.6, 1.4]$, accommodating cameras with different dynamic ranges.
  - **Random saturation**: scaling factor in $[0.6, 1.4]$ on RGB inputs, modeling color intensity changes.
  - **JPEG compression artifacts**: random quality factor in $[50, 90]$, matching the lossy compression in JPEG.

  Sensor augmentations simulate hardware imperfections of low-cost cameras such as the OV7675 (2):
  - **Gaussian noise**: additive noise with standard deviation $\sigma \approx 0.04 \times 255$, simulating CMOS sensor noise in low light conditions.
  - **Gaussian blur**: kernel sizes of 3, 5, or 7, simulating poor focus when the card is held too close or too far.
  - **Motion blur**: directional kernels (horizontal, vertical, diagonal) of size 5, 7, or 9, simulating blur.

– **Average-pooling blur**: a 3×3 pooling operation applied stochastically, providing a lightweight approximation.

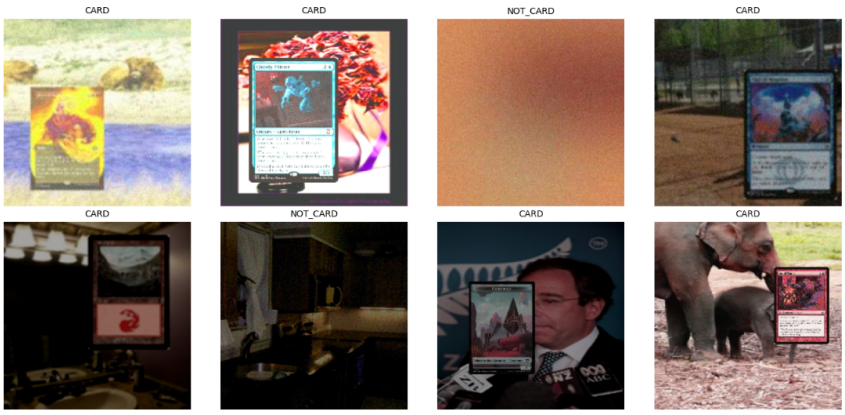

Figure 1: Example images that are generated as a result of the data acquisition pipeline built within Google Collab. Total time to acquire or generate data is roughly 20 minutes per 100,000 photos.

3.2 Preprocessing and model architecture

Images were downsampled to a 96×96 grayscale format to fit the input tensor constraints. We applied a normalization factor of $1/255$ to scale pixel values to the $[0, 1]$ range. Two different CNN architectures were used. Only one was able to be tested on hardware.

1. The CNN architecture was engineered for high-efficiency feature extraction within the memory constraints of the Arduino platform. The specific layers and design rationale are detailed below:
   - **Input Layer:** A $(96, 96, 1)$ grayscale tensor. Single-image inference is enforced, as the TFLM runtime on the Arduino Nano 33 BLE Sense (1) does not support batch processing.
   - **Feature Extraction:**
     – **Conv2D (Layer 1):** Utilizes 8 filters with a 2×2 stride to reduce spatial dimensions from 96×96 to 48×48, utilizing ReLU activation for non-linearity.
     – **Max Pooling:** A 2×2 window downsamples the feature maps to 24×24, utilizing padding to ensure compatibility with the Arduino C++ driver requirements.
     – **Conv2D (Layer 2 & 3):** Subsequent layers extract complex textures and card-specific graphics, progressively downsampling the tensor to 12×12. Each convolutional block is followed by a **Batch Normalization** layer to stabilize activations and accelerate convergence.
   - **Classification Head:**
     – **Global Average Pooling (GAP):** Replaces traditional flattening to reduce the total parameter count and mitigate overfitting.
     – **Dropout:** A rate of 0.3 was applied to the dense features to prevent neuron co-adaptation and improve model robustness.
     – **Dense Output:** A Softmax activation layer provides the final probability distribution.

Training Configuration The model was optimized using the **Adam** adaptive optimizer. We employed **Sparse Categorical Crossentropy (SCC)** as the loss function. While the current task is a binary classification, SCC was selected over Binary Crossentropy due to observed improvements in validation accuracy during experimentation. Furthermore, this architecture provides a scalable foundation for multi-class expansion (e.g., differentiating between *Magic: The Gathering*, *Pokémon*, and *Yu-Gi-Oh!*) by simply adjusting the output dimensionality.

2. Teacher Student Distillation Architecture was designed as a potential method for better accuracy.

   Teacher model. A transfer-learning architecture built on MobileNetV2 (7):

   - **Input**: $224 \times 224 \times 3$ RGB tensor.
   - **Rescaling**: linear remap from $[0, 1]$ to $[-1, 1]$ to match MobileNetV2's expected input range.
   - **Backbone**: MobileNetV2 with ImageNet-pretrained weights and width multiplier $\alpha = 1.0$, trained in two stages, one with frozen for initial head fitting, then top 30 layers unfrozen for fine-tuning at a reduced learning rate.
   - **Pooling**: global average pooling reduces the spatial feature map to a fixed-length vector.
   - **Regularization**: dropout with rate 0.3.
   - **Classifier head**: dense layer with 2 logits, followed by softmax with temperature $T = 4.0$ during distillation.
   - **Parameter count**: approximately 2.3M.

   Student model. A custom depthwise-separable CNN designed to fit within the SRAM budget of the Arduino Nano 33 BLE Sense (1):

   - **Input**: $96 \times 96 \times 1$ grayscale tensor, matching the on-device camera preprocessing.
   - **Stem**: 3×3 convolution with 16 filters, stride 2, followed by batch normalization and ReLU, producing a $48 \times 48 \times 16$ feature map.
   - **Feature blocks**: three depthwise-separable convolution blocks, each consisting of a 3×3 depthwise convolution, batch normalization, ReLU, 1×1 pointwise convolution to expand channels, batch normalization, ReLU, and 2×2 max pooling for spatial downsampling.
   - **Pooling**: global average pooling to produce a 64-dimensional feature vector.
   - **Regularization**: dropout with rate 0.25.
   - **Classifier head**: dense layer with 2 logits, followed by softmax with temperature $T = 4.0$ during distillation and $T = 1.0$ at inference.
   - **Parameter count**: approximately 30K (roughly $77\times$ smaller than the teacher).

Table 1: Channel widths and spatial dimensions for the three depthwise-separable blocks in the student model. Each block applies the same depthwise-pointwise-pool pattern; only the output channel width changes.

Block i	Output channels	Input spatial	Output spatial
1	32	48×48	24×24
2	48	24×24	12×12
3	64	12×12	6×6

The repeated block structure can be expressed compactly. For block $i \in \{1, 2, 3\}$ with input $\mathbf{x}_i$ and target channel width $c_i \in \{32, 48, 64\}$:

$$\mathbf{x}_{i+1} = \mathrm{MaxPool}_{2\times 2}\big(\mathrm{ReLU}\big(\mathrm{BN}\big(\mathrm{Conv}^{c_i}_{1\times 1}(\mathrm{ReLU}(\mathrm{BN}(\mathrm{DWConv}_{3\times 3}(\mathbf{x}_i))))\big)\big)\big)\,. \tag{1}$$

3.3 Quantization

Post-training, we evaluated both 16-bit floating-point (`fp16`) and 8-bit integer (`int8`) quantization to compare their respective impacts on accuracy and model compression. To enable deployment on the microcontroller's integer-only arithmetic units, we utilized a representative dataset to map the floating-point weights and activations to `int8` tensors. This process achieved an approximately $9.8\times$ reduction in model size relative to the baseline while maintaining a high degree of operational accuracy. Finally, the quantized `.tflite` model was converted into a C-style byte array using the `xxd` utility, allowing the model to be compiled directly into the Arduino's Flash memory as a header file. Due to the hex-to-binary encoding overhead and metadata inclusion within the header structure, the final deployed C array occupied 82 KB of flash memory, safely under the 1 MB limit of the Arduino Nano 33 BLE Sense (1).

3.4 Hardware

The primary compute platform for this research is the Arduino Nano 33 BLE Sense (1), a high-performance microcontroller board designed specifically for TinyML applications. It is powered by the Nordic nRF52840 (8) SoC, which features an ARM Cortex-M4 processor running at 64 MHz. The device is equipped with 1 MB of Flash memory and 256 KB of SRAM. Given that the SRAM must accommodate the model's input tensors, weights, and intermediate activations, the severe memory ceiling of 256 KB was a primary driver for our 8-bit quantization strategy. For image acquisition, an OmniVision OV7675 (2) CMOS VGA camera module was interfaced via a serial bus. Although the sensor is capable of 640×480 resolution, the data was downsampled and center-cropped to a 96×96 grayscale format to reduce the memory overhead of the input tensor. The system was powered via a regulated 3.3V supply, with the camera sensor identified as the peak power consumer, drawing approximately 33.67 mA during active frame capture.

Table 2: Relevant hardware specifications for the Arduino Nano 33 BLE Sense (1).

Microcontroller Unit (MCU)	
Component	**Details**
Processor	Arm® Cortex®-M4F (with FPU)
Clock Speed	64 MHz
Flash Memory	1 MB
SRAM	256 KB
Operating Voltage	3.3V (Not 5V tolerant)
Peripherals	GPIO, I2C, PWM
Connectivity	Bluetooth® 5 multiprotocol radio

Table 3: Technical specifications for the OmniVision OV7675 camera module (2).

Image Sensor	
Component	**Details**
Active Array Pixels	640×480 (VGA)
Pixel Size	2.5µm $\times$ 2.5µm
Interface	20-pin DVP
Output Format	RAW, YUV, RGB
Resolution and Frame Rate	160×120 @ 15fps (Target)
Power Requirement Standby	60µW
Power Requirement Active	98 mW

4 Results

The experimental evaluation focuses on quantifying the trade-offs between model complexity, computational latency, and power consumption. These three metrics are critical to evaluating resource constrained machine learning. To establish a performance baseline, the initial Keras float32 model was benchmarked against post-training quantized TFLite variants (Float16 and Int8). On-device verification was conducted on the Arduino Nano 33 BLE Sense (1) using a specialized `arduino_image_provider.cpp` driver to interface with the OV7675 (2) camera module. This setup allowed for an end-to-end analysis of the pipeline, from raw pixel acquisition to final classification. The following sections detail the model's accuracy preservation post-quantization, the energy footprint of the integrated system, and the estimated per-stage latency breakdown of the inference cycle.

Table 4: Comparison of classification accuracy and memory footprint across different quantization levels for the card detection model.

Performance Metrics Comparison			
Metric	Baseline (Float32)	Quantized (Float16)	Quantized (Int8)
Accuracy (%)	99.55	99.54	99.45
Accuracy Drop (%)	—	0.01	0.10
Model Size (KB)	125.89	18.51	12.86
Compression Ratio	1.0×	6.8×	9.8×

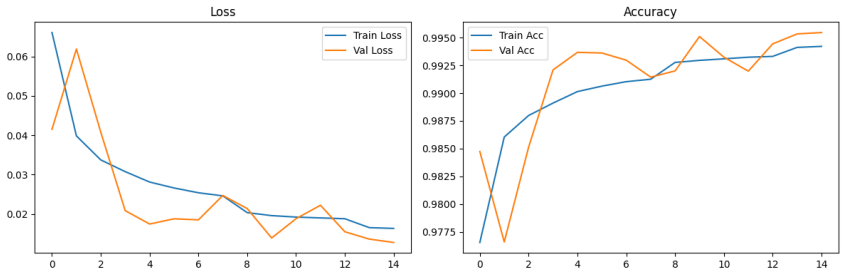

Figure 2: Baseline model training and validation loss/accuracy curves over 15 epochs. The model achieves high convergence with no evidence of overfitting. While some minor volatility is observed in the validation metrics, the overall trend confirms stable learning and strong generalization.

Table 5: Estimated power consumption breakdown per hardware component during active inference at 3.3V, highlighting the camera sensor as the primary power draw.

Estimated Power Consumption Summary		
Component	Est. Current (mA)	Est. Power (mW)
Processor	~2.24	~7.39
Camera	~33.67	~98.00
LEDs	~2.00	~6.60
Total	**~37.91**	**~111.99**

Table 6: Computational latency breakdown for a single inference cycle, including serial data acquisition, image preprocessing, and 8-bit integer neural network execution.

System Latency Summary	
Task	Estimated Latency
Camera	~150.0 ms
Preprocessing	~21.6 ms
Inference	~187.5 ms
Total	**~359.1 ms**

5 Discussion

The primary objective was the deployment of a real-time card detection system on the Arduino Nano 33 BLE Sense (1). To optimize performance, we evaluated several methodologies, including student-teacher architectures and both 8-bit and 16-bit post-training quantization. Although initial experiments with diverse negative datasets and model variants resulted in significant accuracy fluctuations, the final optimized models achieved convergence at 99% validation accuracy. Critical technical hurdles included the hardware-level integration of the OV7675 (2) camera module, model compatibility with the restricted operator set of the TensorFlow Lite for Microcontrollers (TFLM) framework (4), and ensuring the Arduino's input logic manually replicated the quantization handshake:

$$(pixel/scale) + zero\_point$$

.

6 Conclusion

This research demonstrates the feasibility of deploying high-granularity, real-time object classification on highly resource-constrained edge hardware. By utilizing a hybrid data augmentation strategy and optimizing a 2D CNN for 8-bit integer quantization, we successfully developed a Magic: The Gathering card detection system that operates within the 256 KB SRAM limits of the Arduino Nano 33 BLE Sense (1). Our final model achieved a minimal memory footprint of 82 KB while maintaining a validation accuracy of 99.45%, proving that specialized TinyML frameworks can replace expensive, PC-dependent hardware for niche computer vision tasks.

Future work will focus on improving the inference latency, which is currently bottlenecked by the serial data acquisition and image preprocessing stages. Implementing hardware-level windowing and exploring more aggressive pruning techniques could further reduce the 359.1 ms inference cycle, potentially enabling the detection of multiple cards in a single frame. This project serves as a scalable template for hobbyist and industrial inventory automation using low-cost, off-the-shelf microcontrollers.

References

[1] Arduino SA. "Arduino Nano 33 BLE Sense Documentation." Arduino. Available at: `https://docs.arduino.cc/hardware/nano-33-ble-sense`. Accessed April 20, 2026.

[2] OmniVision Technologies. *OV7675 CMOS VGA (640x480) CameraChip Sensor Datasheet*. OmniVision Technologies, Inc., Santa Clara, CA, 2010.

[3] P. Warden and D. Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, Sebastopol, CA, 2019.

[4] TensorFlow Authors. "TensorFlow Lite for Microcontrollers." GitHub Repository. Available at: `https://github.com/tensorflow/tflite-micro/tree/main/tensorflow/lite/micro/examples/person_detection`. Accessed April 1, 2026.

[5] Scryfall LLC. "Scryfall API Documentation." Scryfall. Available at: `https://scryfall.com/docs/api`. Accessed April 10, 2026.

[6] T. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. "Microsoft COCO: Common Objects in Context." *Proceedings of the European Conference on Computer Vision (ECCV)*. Available at: `https://cocodataset.org/`. Accessed April 15, 2026.

[7] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. C. Chen. "MobileNetV2: Inverted Residuals and Linear Bottlenecks." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018. Available at: `https://arxiv.org/abs/1801.04381`.

[8] Nordic Semiconductor. *nRF52840 Product Specification v1.7*. Nordic Semiconductor ASA, Trondheim, Norway, 2019. Available at: `https://infocenter.nordicsemi.com/pdf/nRF52840_PS_v1.7.pdf`. Accessed April 22, 2026.

A Technical appendices and supplementary material

The complete source code for this project, including the 2D CNN architecture, 8-bit quantization scripts, and the hybrid augmentation pipeline, is available on GitHub at the following repository:

`https://github.com/gregblast-bot/AutoMagic_Detection`

A.1 Source Code and Datasets

The GitHub link above contains:

- **Training Pipeline:** Jupyter Notebooks detailing the transition from the Scryfall API data to the final student/teacher and quantized models.
- **Camera Test:** The C++ source code for using an OV7675 (2) camera module with the Arduino Nano 33 BLE Sense (1) and a Python script to view a live feed over a serial link.
- **TFLM Implementation:** The C++ source code to detect cards with the Arduino Nano 33 BLE Sense (1).
- **Output:** Screenshots of relevant metrics and model output.

A.2 Additional Results

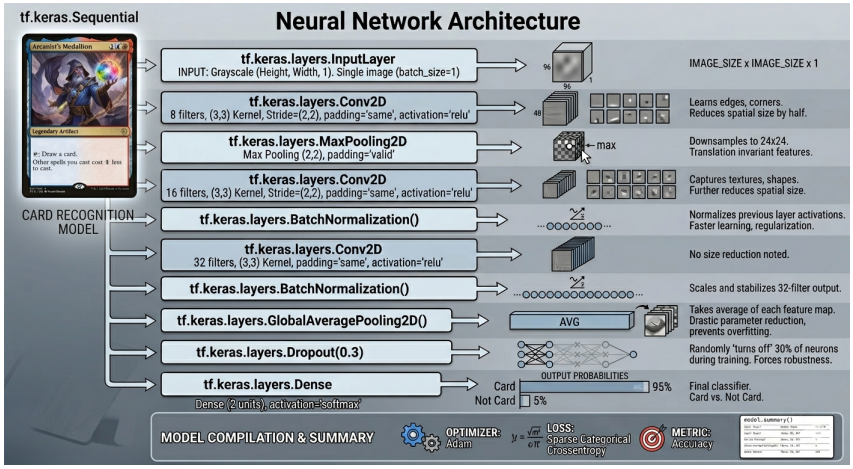

Figure 3: Baseline model used for the Arduino Nano 33 BLE Sense (1).

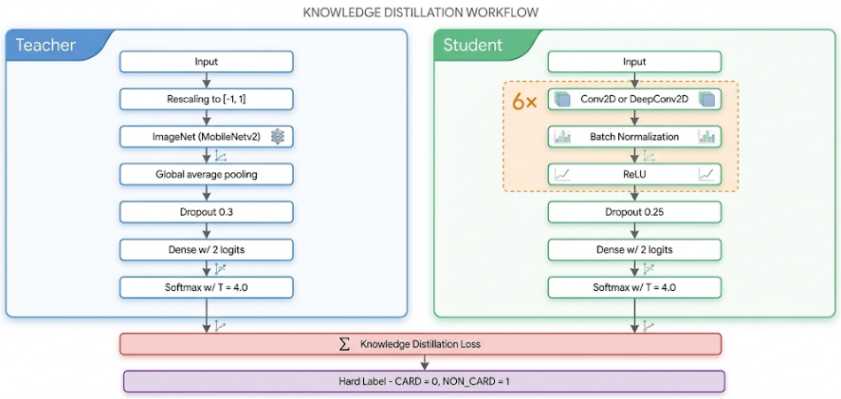

Figure 4: Student-teacher architecture to be tested with the embedded device in the future.

Figure 5: False positive on a Pokemon card over a Magic the Gathering Card - this is due to similar shape and image of these cards.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction clearly outline the development of a specialized TinyML CNN for Magic: The Gathering card detection on the Arduino Nano 33 BLE Sense.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Limitations regarding environment-specific constraints are discussed in the Discussion section.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This paper is focused on empirical system design and implementation and does not contain new theoretical theorems or proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: The Methodology section details the architecture, preprocessing steps, and quantization parameters.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material.

Answer: [Yes]

Justification: All code for model training, quantization, and the Arduino deployment are included in the

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Detailed hyperparameter settings and data synthesis strategies are provided in the Methodology and Experiments sections.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

   Justification: We focus on deterministic metrics such as model size and latency. However, we report accuracy deltas across quantization levels.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).

- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: This paper specifies the target hardware and relevant memory constraints.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: This work follows all ethical guidelines regarding data usage and disclosure.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer:[Yes],

    Justification: We briefly discuss the benefit of low-cost, portable automation for hobbyists and we mention the broader context of TinyML accessibility.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.

- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The released model is a specialized image classifier for Magic: The Gathering cards and poses no foreseeable risk for misuse or dual-use.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: We credit the Scryfall API developers for ease of access to card images and the TFLM framework developers for the bulk of the Arduino code.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.

- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The card-specific CNN architecture is documented in the paper and the code comments.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: No crowdsourcing or human subjects were used in this research.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: No human subjects were involved in this research.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.

- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer:b[Yes]

    Justification: LLM use is described for rapid prototyping and asssstance with the OV7675 camera setup.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.